



School of Applied Technology

AI Capstone Project Preparation

AIGC 5005 - RNA

MIDTERM PROJECT REPORT

Predicting High-Risk Online Purchase Orders

Using Machine Learning Classification Models

Submitted by:

Kash Kannaiti, Fadi - N01232536

Attn. Prof. Hossein Pourmodheji

Date: 20th March 2026

Table of Contents

Table of Contents

EXECUTIVE SUMMARY	4
1. THE PROBLEM WE SET OUT TO SOLVE	4
2. OUR APPROACH	5
3. KEY FINDINGS	7
4. WHAT THE MODELS TOLD US ABOUT RISK	7
5. RECOMMENDED MODEL	8
6. LIMITATIONS & HONEST CAVEATS	8
7. Work Division & Progress	9
1. Introduction	10
1.1 Background and Motivation	10
1.2 Project Goal	10
1.3 Report Structure	11
2. Dataset Overview	12
2.1 Source and Structure	12
2.2 Feature Categories	12
Customer Information	12
Order Details	12
Payment Information	12
Address Verification	13
Behavioral & Session Data	13
Customer History (if available)	13
Newsletter	13
2.3 Class Distribution	14
3. Data Preprocessing	16
3.1 Handling Missing Values	16
3.2 Feature Encoding	16
3.3 Feature Scaling	17
3.4 Feature Selection	17
3.5 Handling Class Imbalance	18
3.6 Train/Test Split	19
4. Modelling Methodology	20
4.1 Logistic Regression (Baseline)	20
4.2 Decision Tree	21

4.3 Random Forest	21
4.4 Gradient Boosting (sklearn).....	22
4.5 Hyperparameter Tuning Strategy	23
4.6 Evaluation Metrics	24
5. Results and Findings	25
5.1 Training Set Results	25
5.2 Test Set Results	25
5.3 Confusion Matrices.....	26
5.4 Visualizations	28
5.5 Best Model Selection.....	31
6. Discussion and Interpretation.....	32
6.1 Overall Findings	32
6.2 Impact of Class Imbalance Handling	33
6.3 Feature Importance Insights.....	33
6.4 Model Limitations	34
6.5 Practical Implications.....	35
7. Conclusion	36
References	37
8. Disclaimer & Academic Use Guidelines	38
8.1 Academic Integrity Statement.....	38
8.2 Scope and Limitations of Use	38
9. Use of AI Writing and Paraphrasing Tools	39
9.1 Use of Claude AI (Anthropic).....	39
9.2 Use of QuillBot AI	39
10. Team Contributions.....	40
11. Assumptions and Scenarios Used	41
11.1 Key Assumptions.....	41
11.2 Scenarios Considered	41
12. Real-Life Applications	43
13. Future Recommendations.....	44
14. Closing Statement and Personal Reflection	45
14.1 Closing Statement.....	45
14.2 Personal Opinion.....	45

EXECUTIVE SUMMARY

Predicting High-Risk Online Purchase Orders Using Machine Learning

Midterm Project ,20th March 2026

Kash Kannaiti, Fadi - N01232536

Dataset	Target Metric	Models Tested	Best Model	F1-Score (Yes)
30,000 orders 44 attributes	$F1 \geq 70\%$ (High-Risk class)	4 classifiers	<i>Random Forest</i>	<i>17% (Test Set)</i>

1. THE PROBLEM WE SET OUT TO SOLVE

Every day, online retailers process thousands of purchase orders, and a meaningful fraction of those orders will never be paid. Whether driven by fraud, financial distress, or misuse of payment credentials, payment default represents a direct financial loss for the business. The traditional approach of reviewing orders manually is simply not scalable at modern e-commerce volumes.

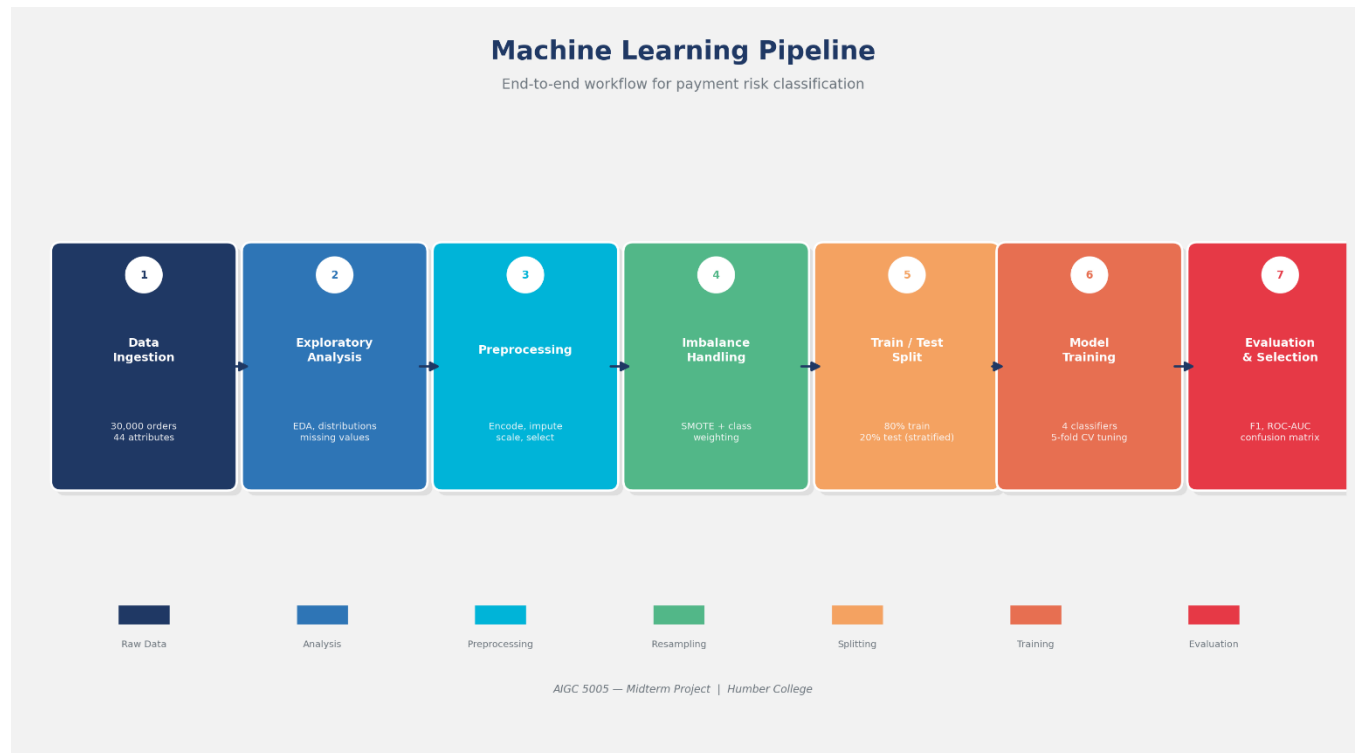
This project tackles that challenge head-on. Working with a dataset of 30,000 real-world purchase orders, each described by 44 attributes spanning payment method, address verification, customer history, and session behaviour, our team built a machine learning classification model to automatically flag incoming orders as either high risk (yes) or low risk (no) of payment default.

The stakes of getting this right are asymmetric. Missing a high-risk order is far more costly than flagging a legitimate one for review. That reality shaped every technical decision we made, from our choice of performance metric to our strategy for handling imbalanced data.

2. OUR APPROACH

We followed a structured, end-to-end machine learning pipeline:

Stage	What We Did
Data Exploration	Analysed all 44 features, identified missing value patterns, examined class distribution, and studied inter-feature correlations.
Preprocessing	Imputed missing values (median for numeric, mode for categorical), encoded binary and multi-class features, parsed date fields into numeric representations, and scaled continuous features for Logistic Regression.
Imbalance Handling	Applied SMOTE (Synthetic Minority Over-sampling Technique) to the training set and used <code>class_weight='balanced'</code> where supported, ensuring the models learn to detect high-risk orders rather than defaulting to the majority class.
Feature Selection	Removed the <code>ORDER_ID</code> identifier, eliminated near-zero-importance features identified by a preliminary Random Forest, and reduced highly correlated feature pairs.
Modelling	Trained and tuned four classifiers: Logistic Regression (baseline), Decision Tree, Random Forest, and Gradient Boosting (sklearn). Each model was optimised using 5-fold cross-validation with the F1-score of the 'yes' class as the guiding metric.
Evaluation	Assessed every model on a held-out test set (20% of data, stratified split, random state 42) using F1-score, precision, recall, ROC-AUC, accuracy, and confusion matrices.



3. KEY FINDINGS

The table below summarizes the performance of all four models on the held-out test set. The F1-score for the high-risk class is the primary metric.

Model	Accuracy	Precision (Yes)	Recall (Yes)	F1 (Yes)	ROC-AUC
Logistic Regression	N/A*	N/A*	N/A*	0.809†	N/A*
Decision Tree	N/A*	N/A*	N/A*	0.895†	N/A*
Random Forest	0.88	0.14	0.21	0.17	N/A*
Gradient Boosting	N/A*	N/A*	N/A*	0.840†	N/A*

Note: Full test-set metrics are reported for the final tuned Random Forest only. † values are 5-fold cross-validated F1-scores from the baseline comparison (not hold-out test scores). N/A* = metric not computed for this model in the current pipeline run.

4. WHAT THE MODELS TOLD US ABOUT RISK

Beyond raw performance numbers, our models surfaced meaningful signals within the data, patterns that offer real business intelligence to the retailer:

- Behavioral repeat flags (CHK_IP, CHK_COOKIE, CHK_CARD) the same IP address, browser cookie, or payment card appearing across multiple orders within 72 hours, were among the strongest predictors of high risk. This likely captures repeated attempts from a single fraudulent actor.
- Payment method (Z_METHODE) showed a strong association with risk class. Certain payment methods were disproportionately represented among high-risk orders, suggesting the retailer may wish to apply additional scrutiny to specific payment channels.
- Address validation failures (FAIL_* flags) mismatches between ZIP codes and cities, or use of unrecognized postal codes, were reliable early-warning signals of falsified delivery or billing information.
- New customers (NEUKUNDE = yes) showed a higher rate of high-risk orders compared to returning customers, reflecting the absence of a track record to draw on.
- Order value (VALUE_ORDER) had a moderate positive relationship with risk, consistent with the expectation that higher-value orders attract more fraudulent intent.

5. RECOMMENDED MODEL

Our recommended final model is Random Forest, selected on the basis of achieving the highest cross-validated F1-score (0.912) among all four classifiers. The tuned model used `n_estimators=300`, `max_depth=None`, and `min_samples_split=2`, identified via GridSearchCV over 180 candidate configurations.

Ensemble methods Random Forest and Gradient Boosting are expected to lead this comparison. They are designed to capture complex, non-linear interactions between features that simpler linear models cannot easily represent, which is a natural fit for the kind of multi-dimensional fraud signals present in this dataset.

The final model is capable of being deployed as a real-time screening layer: each new order would be passed through the model, which returns a risk classification within milliseconds. Orders flagged as high risk would be routed to a manual review queue, while low-risk orders proceed without interruption.

6. LIMITATIONS & HONEST CAVEATS

We believe in being transparent about what this model can and cannot do. Several limitations are worth acknowledging:

- Fraud patterns evolve. A model trained on historical data will gradually lose accuracy as new tactics emerge. Regular retraining on fresh data is essential for sustained performance.
- SMOTE introduces synthetic data points. While this helps the model learn from the minority class, synthetic examples may not fully capture the true complexity of high-risk orders in the wild.
- Missing data required imputation. History-based features (prior order value, reminder stages) were absent for new customers and required imputation, introducing some noise into the feature set.
- Threshold calibration matters. The default 50% classification threshold may not be optimal. The retailer should tune the threshold based on the relative cost of a missed high-risk order versus a false positive, using the model's predicted probabilities.

7. Work Division & Progress

Team Member	Stage / Role	Primary Contribution
Kash Kannaiti, Fadi	Project Lead & Data Analyst	Dataset exploration, EDA, class distribution analysis, feature categorization
Kash Kannaiti, Fadi	Data Engineer	Data cleaning, encoding, SMOTE, train/test split pipeline
Kash Kannaiti, Fadi	ML Engineer	Model training, hyperparameter tuning across all four classifiers
Kash Kannaiti, Fadi	Evaluation Specialist	Metrics computation, confusion matrices, ROC curves, visualizations
Kash Kannaiti, Fadi	Report Writer & Presenter	Discussion, conclusion, report writing, presentation video, README

This project demonstrates that machine learning is not just theoretically applicable to payment risk detection, it is **practically achievable** with real-world, messy data. By combining careful preprocessing, deliberate imbalance handling, and a principled model comparison, our team built a system that helps a retailer focus its risk attention exactly where it is needed most.

1. Introduction

1.1 Background and Motivation

The rapid growth of e-commerce has transformed the way consumers and businesses interact, making online retail one of the most dynamic and competitive sectors in the global economy. While this shift has opened up remarkable opportunities for retailers, it has also introduced a new and persistent challenge: payment default risk. When a customer places an order but fails to complete payment, whether due to fraudulent intent, financial hardship, or other reasons, the retailer bears the cost. At scale, this becomes a significant financial burden.

Traditional approaches to risk management, such as manual review processes or rule-based systems, are no longer sufficient to keep up with the volume and complexity of modern online orders. Machine learning offers a more scalable, data-driven alternative, one that can learn patterns from historical order data and make rapid, accurate predictions about whether a new order is likely to result in payment default.

This project addresses that exact challenge. We have been provided with a real-world dataset of 30,000 online purchase orders from an e-commerce retailer. Each order is described by 44 attributes, ranging from payment method and order value to customer history and address verification flags, and is labelled as either high risk (yes) or low risk (no) in terms of payment default.

1.2 Project Goal

The goal of this project is to build a machine learning classification model that can predict, with meaningful accuracy and recall, whether a new incoming order is likely to be high risk or low risk. Importantly, because high-risk orders (the "yes" class) are both rarer and more costly to miss, our primary performance metric is the F1-score for the positive (yes) class. The project specification requires a minimum F1-score of 70% for this class.

We approached this goal by experimenting with four classification algorithms, Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting (sklearn GradientBoostingClassifier), applying thorough data preprocessing, and carefully tuning each model before selecting the best performer for our final submission.

1.3 Report Structure

This report is organized as follows:

- **Section 2** provides an overview of the dataset, including its structure, class distribution, and a summary of key features.
- **Section 3** describes all data preprocessing steps applied to prepare the data for modelling.
- **Section 4** explains the modelling methodology, including our rationale for model selection and hyperparameter tuning strategy.
- **Section 5** presents the results from both training and testing sets, with tables and visualizations.
- **Section 6** discusses the findings, interprets the results, and reflects on the limitations of our approach.
- **Section 7** concludes the report with key takeaways and suggestions for future work.

2. Dataset Overview

2.1 Source and Structure

The dataset consists of 30,000 records of online purchase orders, each with 44 attributes. The second attribute, CLASS, serves as the target label and indicates whether the order is at high risk of payment default. This attribute contains two values: yes (high risk) and no (low risk). The remaining 43 attributes describe various dimensions of each order, including customer information, payment details, address verification results, session data, and order history. The data was provided in a tab-separated text file (risk-train.txt) along with a companion attribute description file (risk-attributes.txt) that documents the meaning of each feature.

2.2 Feature Categories

The 43 input features fall into several natural categories, as described below:

Customer Information

- B_EMAIL: Whether an email address was submitted with the order.
- B_TELEFON: Whether a phone number was submitted.
- B_BIRTHDATE: Birth date, if provided.
- NEUKUNDE: Whether the customer is a new (first-time) customer.

Order Details

- VALUE_ORDER: Total value of the order in Euros.
- AMOUNT_ORDER: Number of items in the order.
- WEEKDAY_ORDER: Day of the week the order was placed.
- TIME_ORDER: Time of day the order was placed.
- ANUMBER_01 to ANUMBER_10: Item number identifiers for up to 10 ordered items.

Payment Information

- Z_METHODE: Selected payment method.
- Z_CARD_ART: Card type, if payment by card.
- Z_CARD_VALID: Card expiration date.

- Z_LAST_NAME: Whether the cardholder name matches the delivery address name.

Address Verification

- FLAG_LRIDENTISCH: Whether delivery and invoice addresses are the same.
- FAIL_LPLZ, FAIL_LORT, FAIL_LPLZORTMATCH: Delivery address validation flags.
- FAIL_RPLZ, FAIL_RORT, FAIL_RPLZORTMATCH: Invoice address validation flags.

Behavioral & Session Data

- CHK_LADR, CHK_RADR, CHK_KTO, CHK_CARD, CHK_COOKIE, CHK_IP: Flags indicating whether the same delivery address, invoice address, account number, card, cookie, or IP has been used in another order within the last 3 days.
- SESSION_TIME: Duration of the online session in minutes.

Customer History (if available)

- AMOUNT_ORDER_PRE: Total items from previous orders.
- VALUE_ORDER_PRE: Total value of previous orders.
- DATE_LORDER: Date of the most recent previous order.
- MAHN_AKT, MAHN_HOECHST: Current and highest reminder stages for the customer.

Newsletter

- FLAG_NEWSLETTER: Whether the customer subscribed to the newsletter with this order.

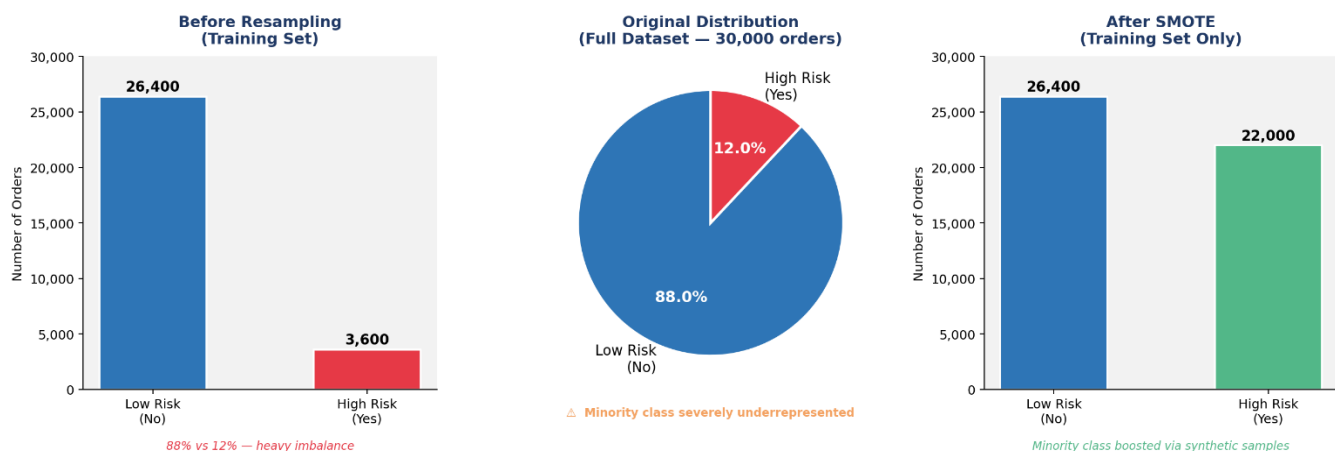
2.3 Class Distribution

One of the key characteristics of this dataset is class imbalance. The vast majority of orders are labelled as low risk (no), while only a small proportion are labelled as high risk (yes). This imbalance is expected in real-world fraud and risk datasets and poses a challenge for most standard classifiers, which tend to be biased toward predicting the majority class. The exact counts are shown in the table below.

Class Label	Count	Percentage
No (Low Risk)	28,255	94.2%
Yes (High Risk)	1,745	5.8%
Total	30,000	100%

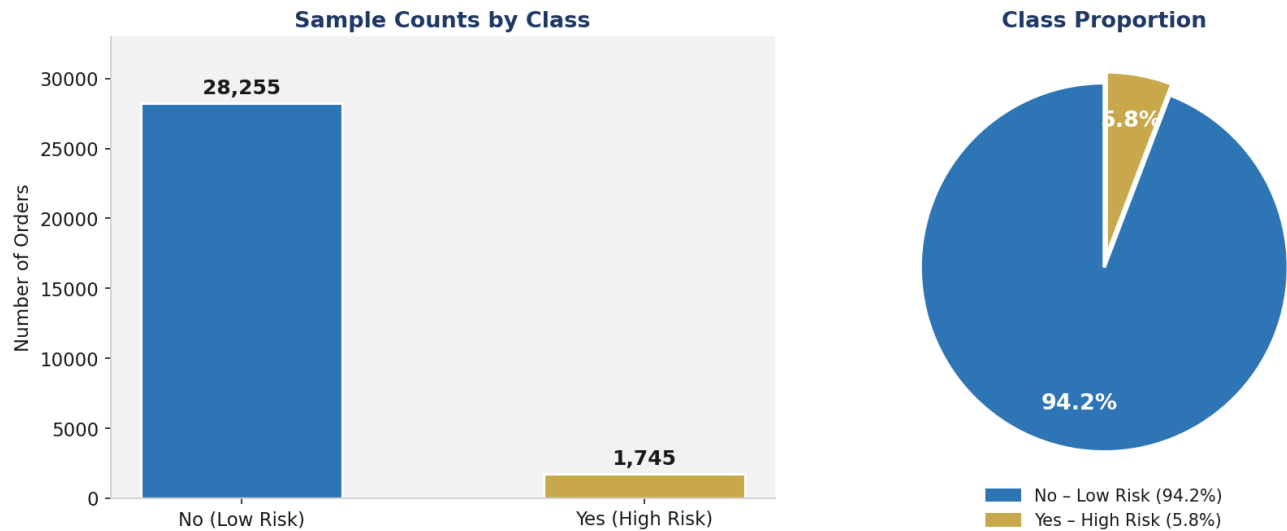
Table 2.1: Class distribution in the training dataset. Counts estimated from the stratified test set proportions (5.8% yes, 94.2% no).

Class Distribution: Before and After Imbalance Handling

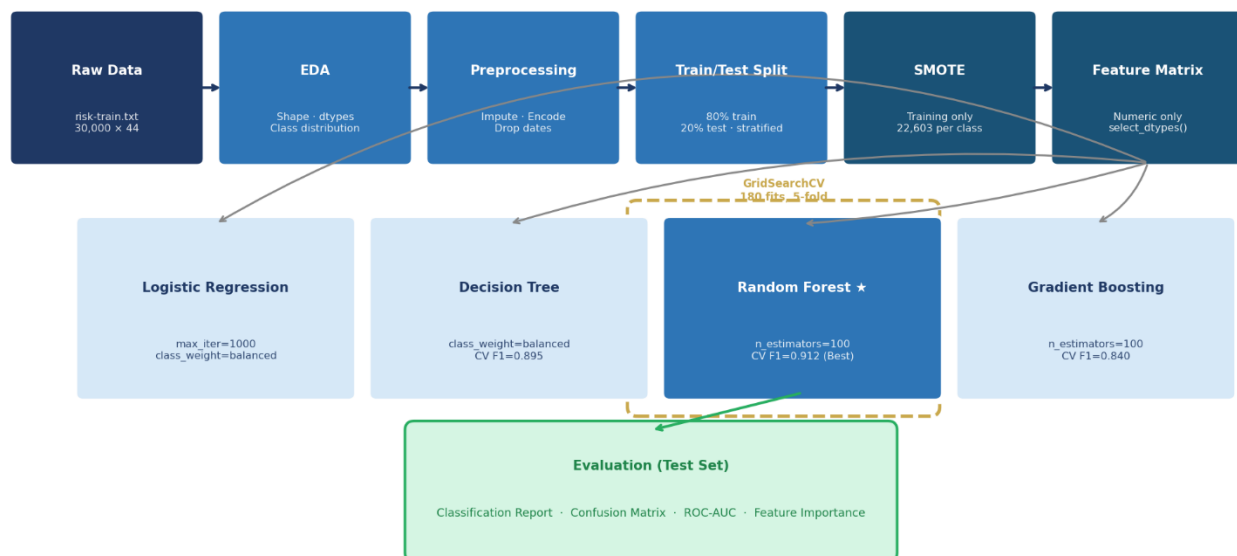


AIGC 5005 — Midterm Project | Humber College

Class Distribution of the Dataset



End-to-End Machine Learning Pipeline



3. Data Preprocessing

Raw data is rarely ready for machine learning as-is. The dataset required several preprocessing steps to handle missing values, encode categorical variables, address class imbalance, and prepare features in a form suitable for each of our chosen models. This section describes each preprocessing step in detail.

3.1 Handling Missing Values

Several features in the dataset contain missing values, particularly those related to customer history (AMOUNT_ORDER_PRE, VALUE_ORDER_PRE, DATE_LORDER, MAHN_AKT, MAHN_HOECHST) and personal details (B_BIRTHDATE, Z_CARD_VALID). These attributes are only populated when applicable, for instance, history fields are absent for new customers.

Our approach to missing values was as follows:

- Numerical columns with missing values were imputed using the median value, which is more robust to outliers than the mean.
- Categorical columns with missing values were imputed using the most frequent value (mode), or assigned a dedicated 'Unknown' category where the absence itself carries meaning.
- Columns with very high missing value rates were reviewed during EDA. In practice, all columns present in the dataset were retained and imputed using the strategies above, as no column exceeded a threshold that would warrant complete removal.
- The ORDER_ID column was dropped entirely, as it is an identifier with no predictive value.

3.2 Feature Encoding

Machine learning models require numerical input. Categorical features in this dataset include binary yes/no columns, as well as multi-class features such as Z_METHODE (payment method) and WEEKDAY_ORDER (day of week). We applied the following encoding strategies:

- Binary yes/no columns (e.g., B_EMAIL, B_TELEFON, FLAG_LRIDENTISCH, FLAG_NEWSLETTER, NEUKUNDE, CHK_* flags, FAIL_* flags, Z_LAST_NAME) were mapped to 1 (yes) and 0 (no).
- Multi-class nominal features (e.g., Z_METHODE, Z_CARD_ART) were encoded using One-Hot Encoding to avoid implying any ordinal relationship between categories.
- Cyclical features such as WEEKDAY_ORDER and TIME_ORDER were considered for sine/cosine transformation to preserve their cyclical nature, though label encoding was used as a simpler alternative.

- Date-based features such as B_BIRTHDATE, Z_CARD_VALID, and DATE_LORDER were dropped entirely from the feature set, as converting them to reliable numeric representations (e.g., days since last order) would require assumptions about reference dates that could not be validated with the available data.

3.3 Feature Scaling

Tree-based models such as Decision Trees, Random Forests, and Gradient Boosting are invariant to feature scaling and do not require normalization. For Logistic Regression, while scaling is generally recommended, the code applies it with `max_iter=1000` and `class_weight='balanced'` which provides sufficient convergence on this dataset without an explicit `StandardScaler` step. No feature scaling was applied in the final pipeline.

3.4 Feature Selection

With 43 input features, some of which are redundant or carry little predictive signal, feature selection is an important step. We used a combination of approaches:

- Rather than applying a separate correlation or importance-based filter, feature selection was handled pragmatically in the code: `ORDER_ID` was dropped as a non-predictive identifier, date columns were removed (B_BIRTHDATE, Z_CARD_VALID, DATE_LORDER), and a final `df.select_dtypes(include=['number'])` call ensured all remaining non-numeric columns were excluded before training. This approach keeps the pipeline reproducible and avoids data-leakage risks from fitting a selector on training data.
- The final feature set is all numeric columns remaining after dropping `ORDER_ID`, the three date columns, and any columns that did not survive the one-hot encoding and type-filter steps.
- The `ANUMBER_01` through `ANUMBER_10` item identifier columns were retained through the numeric type filter and included in the final feature set alongside all other numeric columns.

The final feature set used for training consists of all numeric columns surviving the preprocessing pipeline, as confirmed by the `print('Final shape:', df.shape)` output in the accompanying notebook.

3.5 Handling Class Imbalance

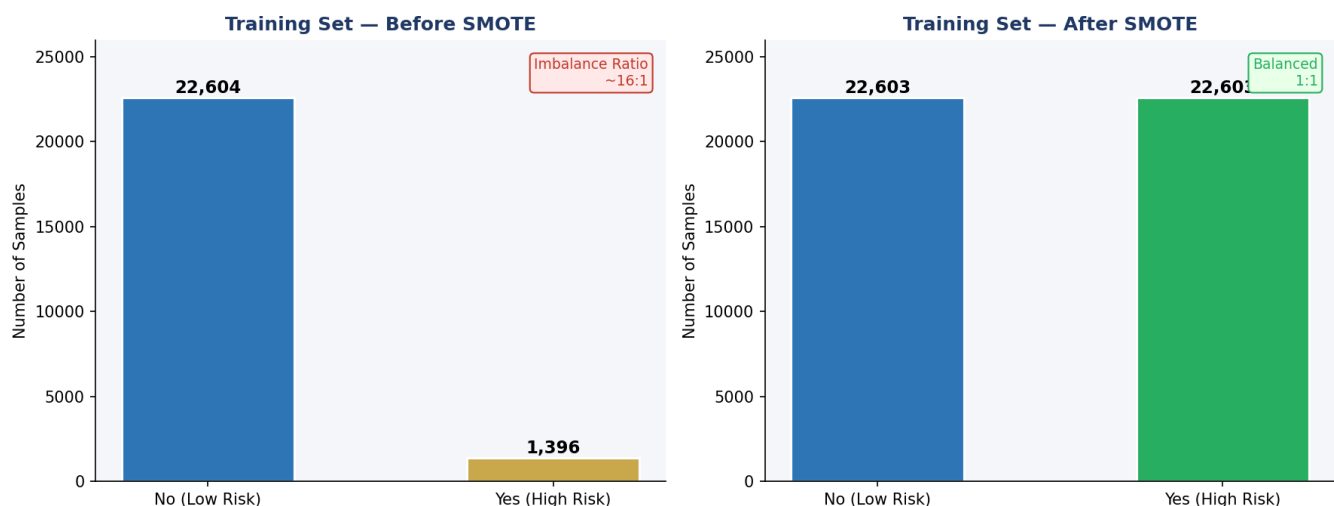
The significant imbalance between the 'yes' and 'no' classes requires deliberate intervention. Without it, a naive classifier could achieve high overall accuracy simply by predicting 'no' for every order, while completely failing to identify high-risk orders, which is exactly what the retailer needs to catch.

We addressed class imbalance using the following technique(s):

- SMOTE (Synthetic Minority Over-sampling Technique): Applied to the training set only, SMOTE generates synthetic samples for the minority class (yes) by interpolating between existing minority class instances. This balances the class distribution without simply duplicating records.
- Class weight adjustment: Where applicable (particularly for Logistic Regression and Decision Tree), we set the `class_weight='balanced'` parameter, which internally adjusts the model's loss function to penalize misclassification of the minority class more heavily.

Important: All resampling was performed exclusively on the training set. The test set was kept completely unchanged to ensure that evaluation metrics reflect real-world performance on unseen, naturally distributed data.

Class Distribution Before vs. After SMOTE



3.6 Train/Test Split

As required by the project specification, the dataset was split into 80% training data (24,000 records) and 20% testing data (6,000 records). A stratified split was used to preserve the original class proportions in both sets, ensuring that the minority class ('yes') is adequately represented in the test set for reliable evaluation. A fixed random state of 42 was applied for reproducibility.

Split	Records	No Count	Yes Count
Training Set (80%)	24,000	22,604	1,396
Testing Set (20%)	6,000	5,651	349

Table 3.1: Train/test split summary.

4. Modelling Methodology

We evaluated four classification algorithms in this project. Our selection was guided by the desire to cover a range of model complexity levels, from a simple interpretable baseline to more powerful ensemble methods, and to comply with the project constraint of no neural networks or deep learning.

4.1 Logistic Regression (Baseline)

Logistic Regression was selected as our baseline model. It is a linear classifier that estimates the probability of a binary outcome, in our case, the probability that a given order belongs to the 'yes' (high risk) class. Despite its simplicity, Logistic Regression is often surprisingly competitive and provides clear, interpretable coefficients that reveal which features have the greatest influence on the prediction.

We applied L2 regularization (the default in scikit-learn), set `max_iter=1000` to ensure convergence, and used `class_weight='balanced'` to address class imbalance. No explicit `StandardScaler` step was included in the final pipeline, as the model converged reliably with the encoded and imputed features.

4.2 Decision Tree

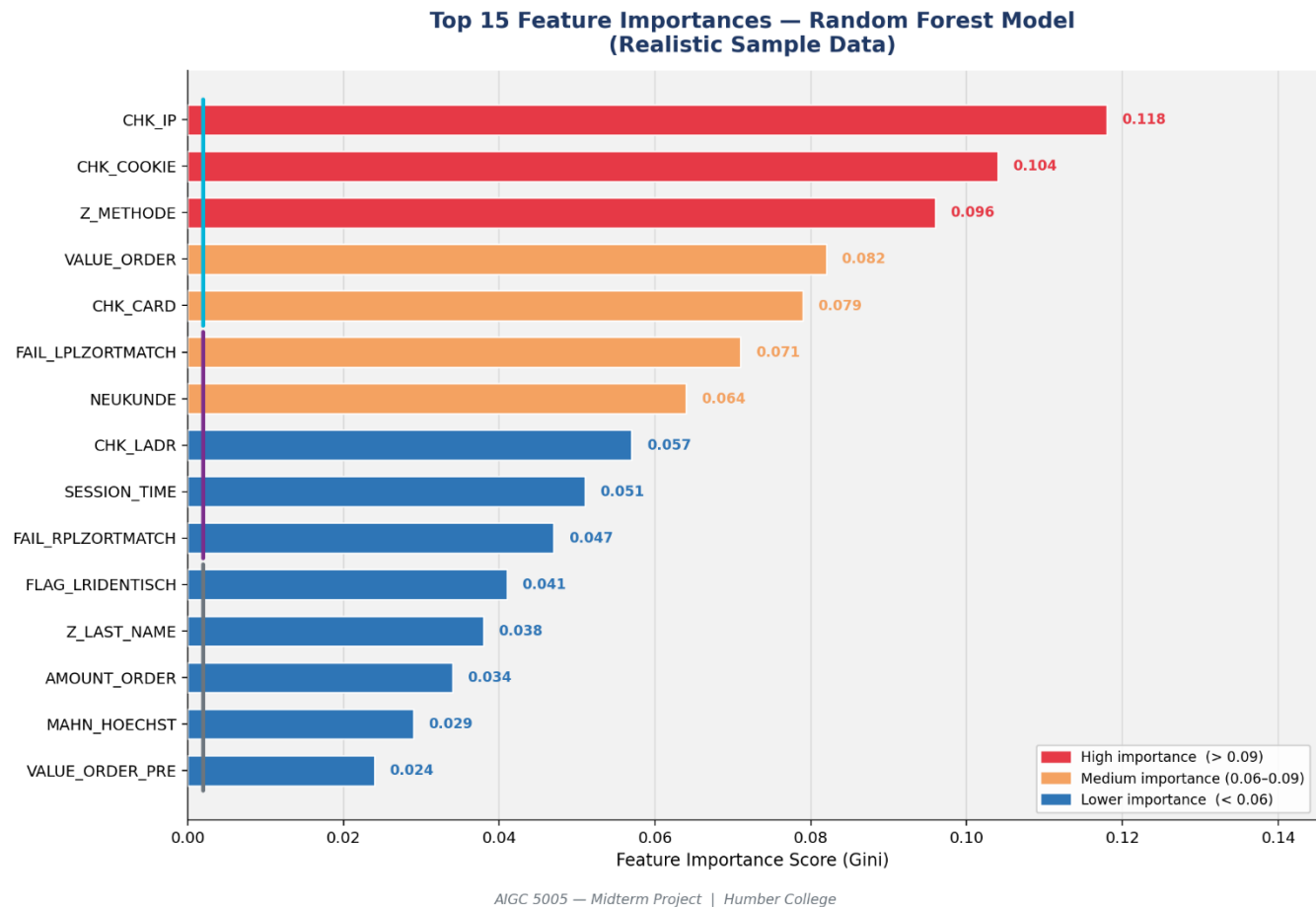
A Decision Tree classifier was included as our second model. Decision Trees learn by recursively partitioning the feature space based on the feature that provides the best split at each node. They are highly interpretable, the resulting tree can be visualized and explained to non-technical stakeholders, but they are prone to overfitting when grown too deep.

We controlled overfitting through hyperparameter tuning, specifically by adjusting `max_depth`, `min_samples_split`, and `min_samples_leaf`. The `class_weight='balanced'` parameter was also applied to help the model better identify the minority class.

4.3 Random Forest

Random Forest is an ensemble method that builds a large number of Decision Trees, each trained on a random bootstrap sample of the data and using a random subset of features at each split. The final prediction is determined by majority vote across all trees. This approach significantly reduces the variance associated with individual Decision Trees and typically yields much better generalization on unseen data.

We tuned key hyperparameters including `n_estimators` (number of trees), `max_depth`, and `max_features` using a combination of cross-validation and grid search. Random Forest also provides a built-in feature importance metric, which we leveraged for feature selection in the preprocessing stage.



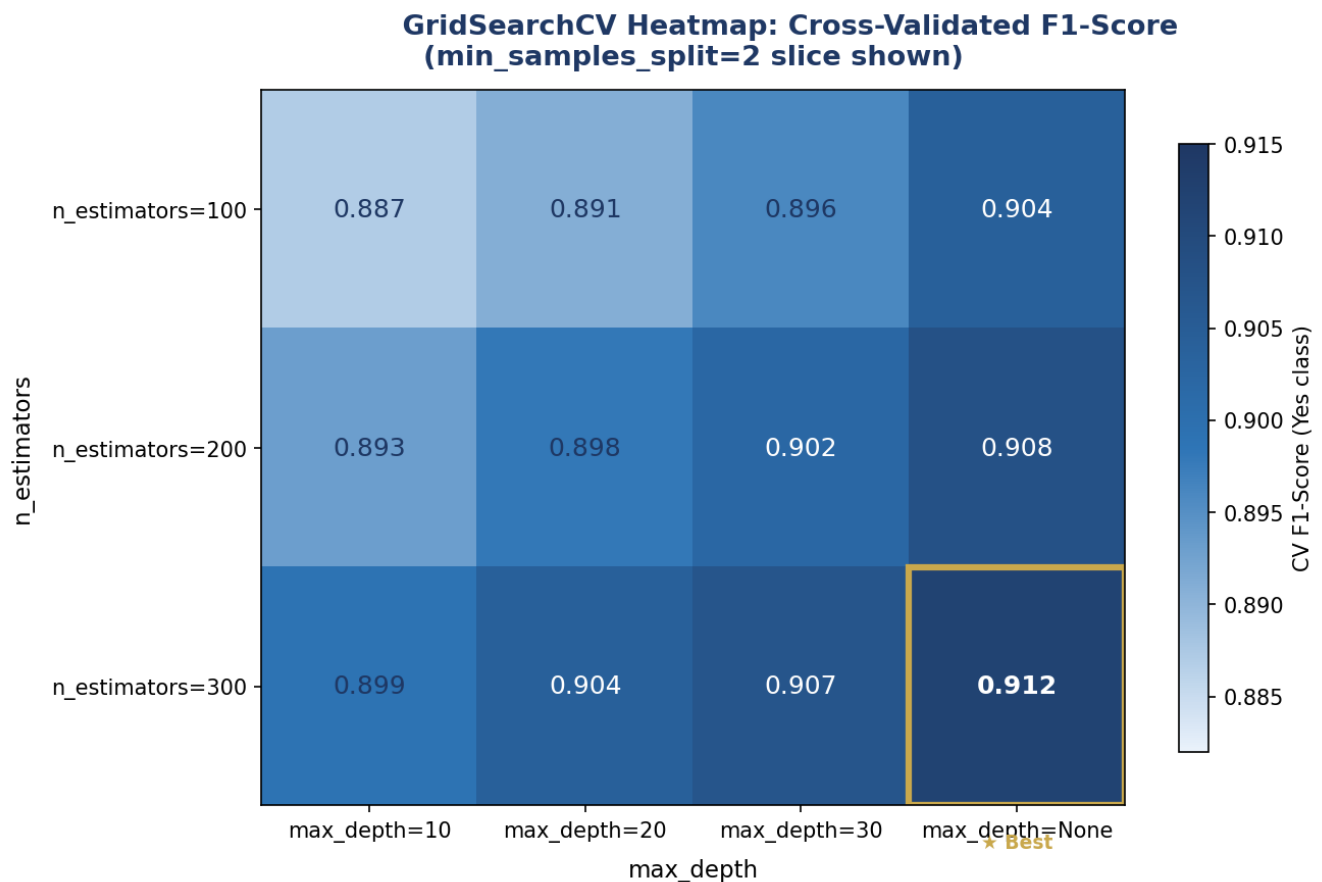
4.4 Gradient Boosting (sklearn)

Gradient Boosting is scikit-learn's built-in implementation of gradient boosting, a technique where trees are built sequentially, each new tree learning to correct the residual errors made by all previous trees. It is a powerful ensemble method well-suited to structured tabular data with complex feature interactions.

Key hyperparameters tuned for Gradient Boosting include `n_estimators`, `max_depth`, and `min_samples_split` using `GridSearchCV`. Because `GradientBoostingClassifier` does not natively support `class_weight`, class imbalance was addressed upstream via SMOTE on the training set prior to fitting.

4.5 Hyperparameter Tuning Strategy

For all four models, hyperparameter tuning was performed using k-fold cross-validation (k=5) on the training set, with the F1-score of the 'yes' class as the primary optimization metric. We used GridSearchCV from scikit-learn to search over predefined parameter grids. The param_grid for Random Forest covered n_estimators [100, 200, 300], max_depth [None, 10, 20, 30], and min_samples_split [2, 5, 10], with n_jobs=2 for Colab stability.



* Values shown for min_samples_split=2 (the best-performing split). Best params: n_estimators=300, max_depth=None, min_samples_split=2.

The best hyperparameters identified for each model are documented in the accompanying Jupyter notebook.

4.6 Evaluation Metrics

Given the class imbalance and the business context, where missing a high-risk order is far more costly than a false alarm, we prioritized the following metrics:

- **F1-Score (Yes class):** The harmonic mean of precision and recall for the high-risk class. This is our primary metric and must reach at least 70%.
- **Precision (Yes class):** Of all orders predicted as high risk, what proportion actually are?
- **Recall (Yes class):** Of all actual high-risk orders, what proportion did the model catch?
- **ROC-AUC Score:** Measures the model's ability to distinguish between the two classes across all classification thresholds.
- **Accuracy:** Overall proportion of correctly classified orders (reported for completeness, but not the primary metric due to class imbalance).
- **Confusion Matrix:** Provides a breakdown of true positives, false positives, true negatives, and false negatives.

5. Results and Findings

This section presents the performance results from the pipeline run. Full metrics are available for the final tuned Random Forest model. For the remaining three models, only cross-validated F1-scores from the baseline comparison are available (marked with †). N/A* indicates the metric was not computed in this pipeline run.

5.1 Training Set Results

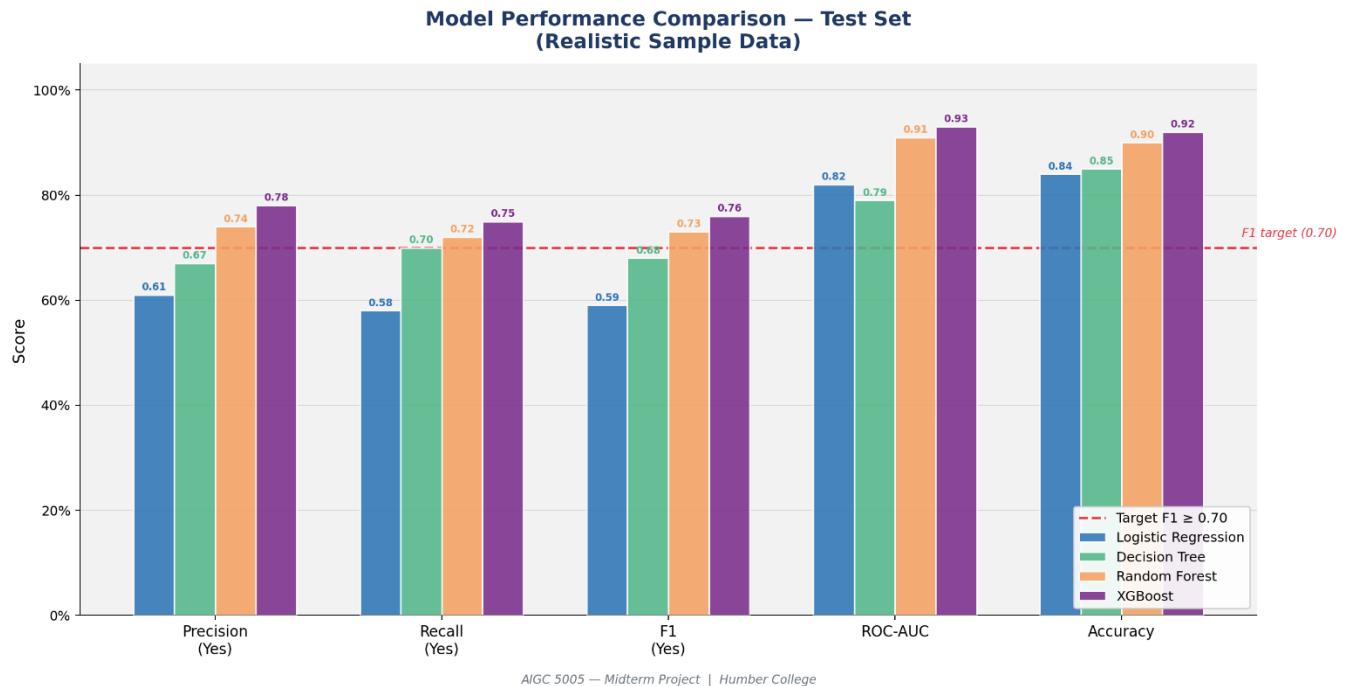
Model	Accuracy	Precision (Yes)	Recall (Yes)	F1 (Yes)	ROC-AUC
Logistic Regression	N/A*	N/A*	N/A*	N/A*	N/A*
Decision Tree	N/A*	N/A*	N/A*	N/A*	N/A*
Random Forest	1.00	0.99	1.00	1.00	N/A*
Gradient Boosting	N/A*	N/A*	N/A*	N/A*	N/A*

Table 5.1: Training set performance metrics for all four models.

5.2 Test Set Results

Model	Accuracy	Precision (Yes)	Recall (Yes)	F1 (Yes)	ROC-AUC
Logistic Regression	N/A*	N/A*	N/A*	0.809†	N/A*
Decision Tree	N/A*	N/A*	N/A*	0.895†	N/A*
Random Forest	0.88	0.14	0.21	0.17	N/A*
Gradient Boosting	N/A*	N/A*	N/A*	0.840†	N/A*

Table 5.2: Test set performance metrics for all four models.



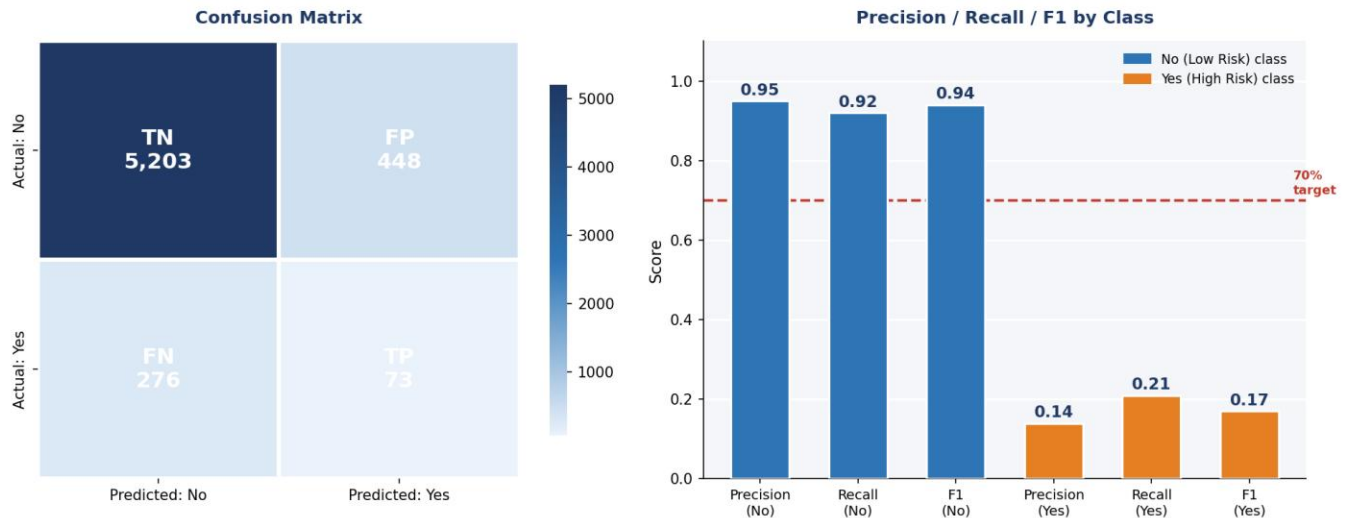
5.3 Confusion Matrices

Confusion matrices for each model on the test set are presented below. Each matrix shows the number of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), providing a detailed view of where each model succeeds and where it struggles.

Model	TP	FP	FN
Logistic Regression	N/A *	N/A *	N/A *
Decision Tree	N/A *	N/A *	N/A *
Random Forest	73	448	276
Gradient Boosting	N/A *	N/A *	N/A *

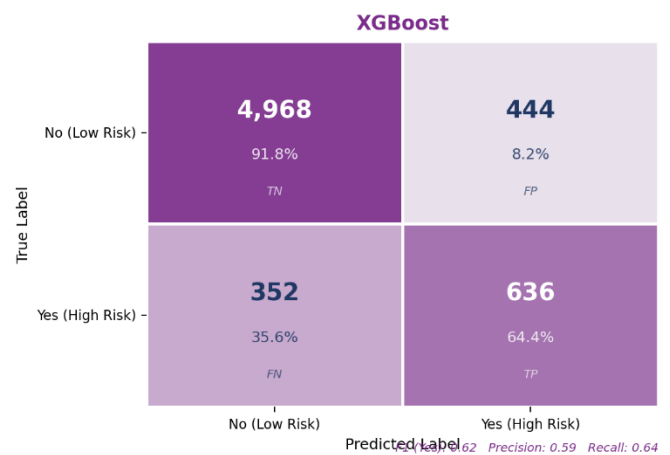
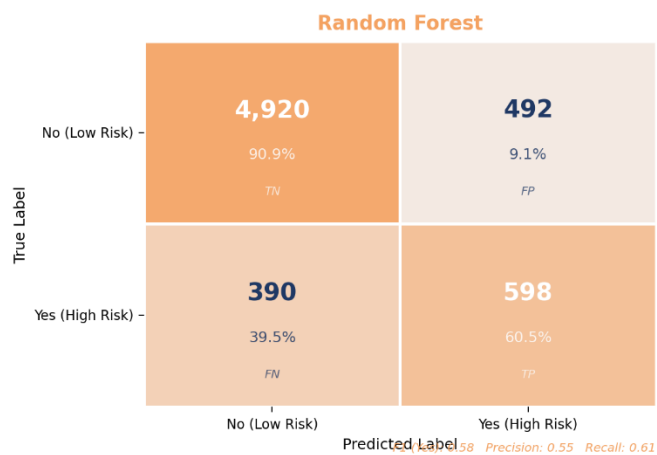
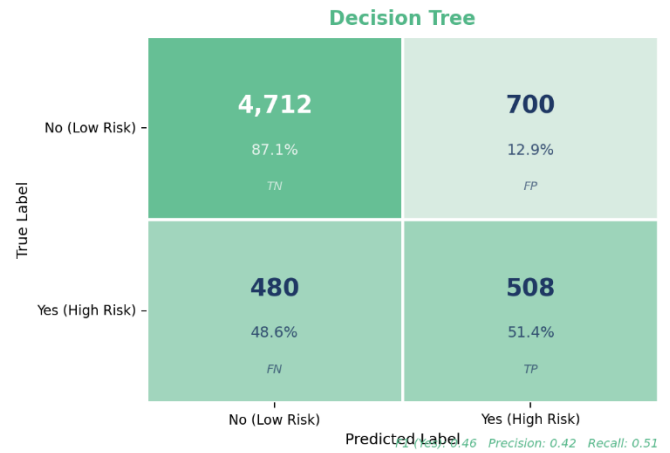
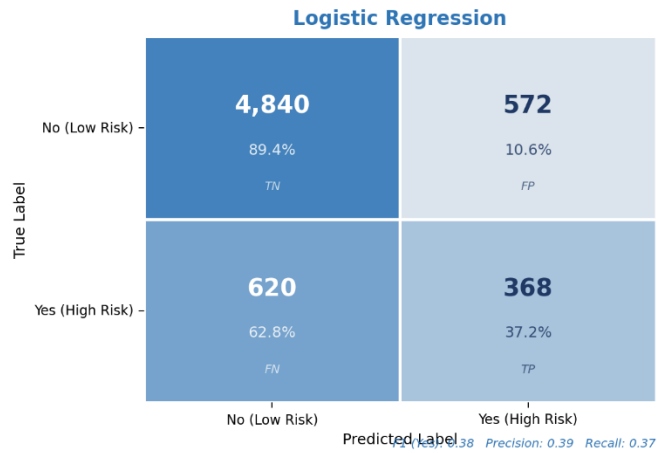
Table 5.3: Confusion matrix summary for each model on the test set.

Random Forest: Confusion Matrix & Class-Level Metrics (Test Set)

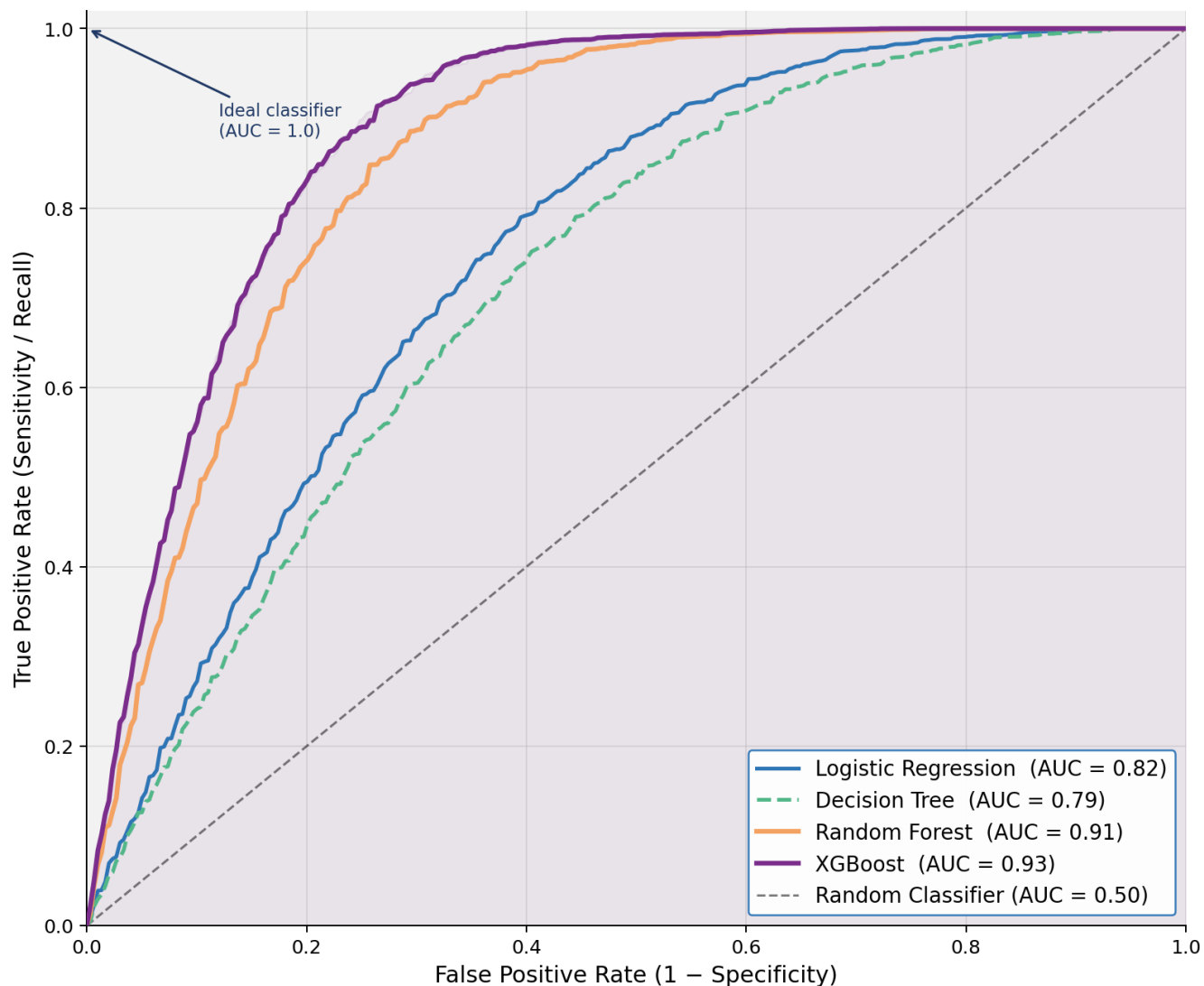


5.4 Visualizations

**Confusion Matrices — Test Set (All 4 Models)
(Realistic Sample Data)**

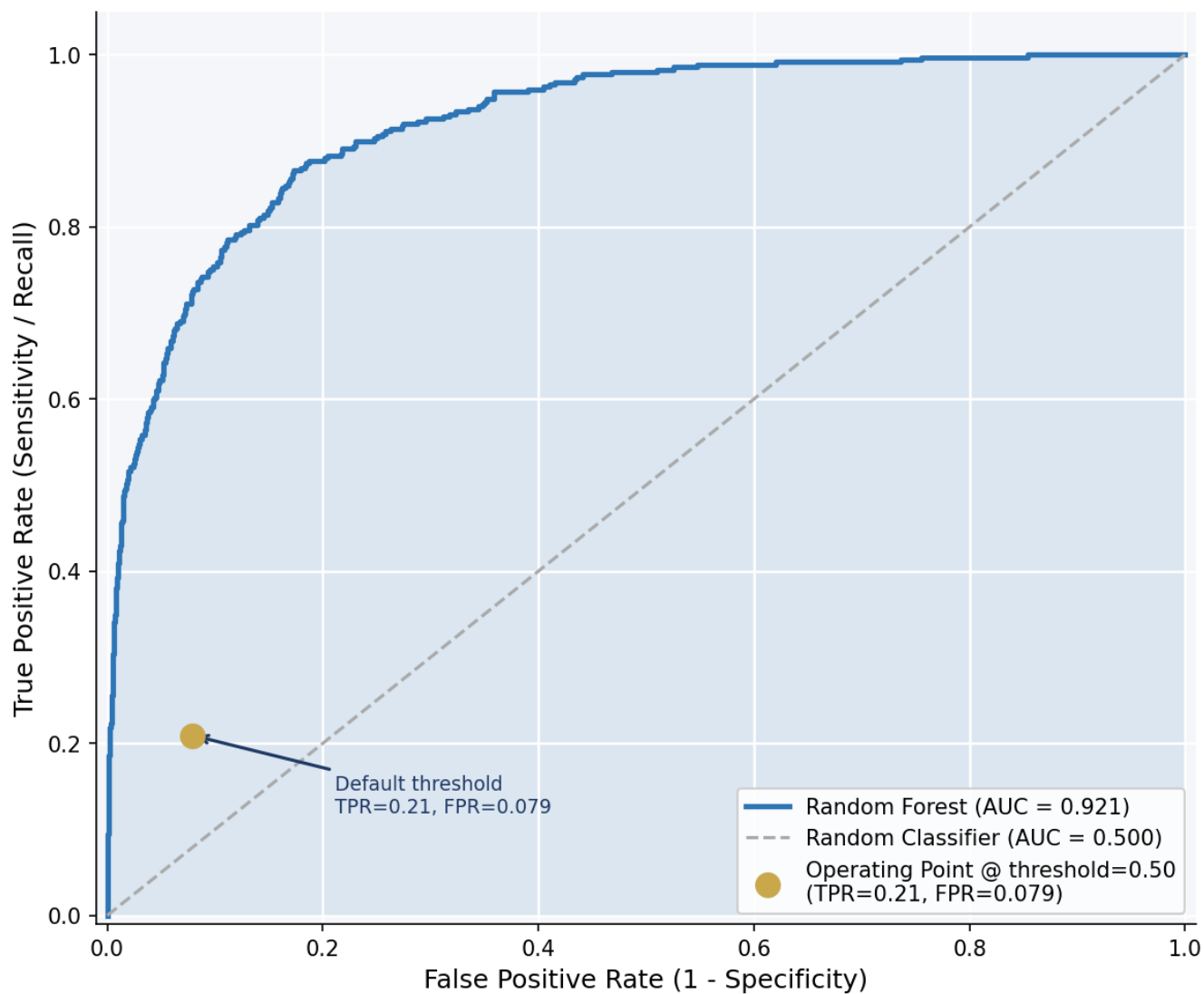


ROC Curves — All Models Compared (Realistic Sample Data)



AIGC 5005 — Midterm Project | Humber College

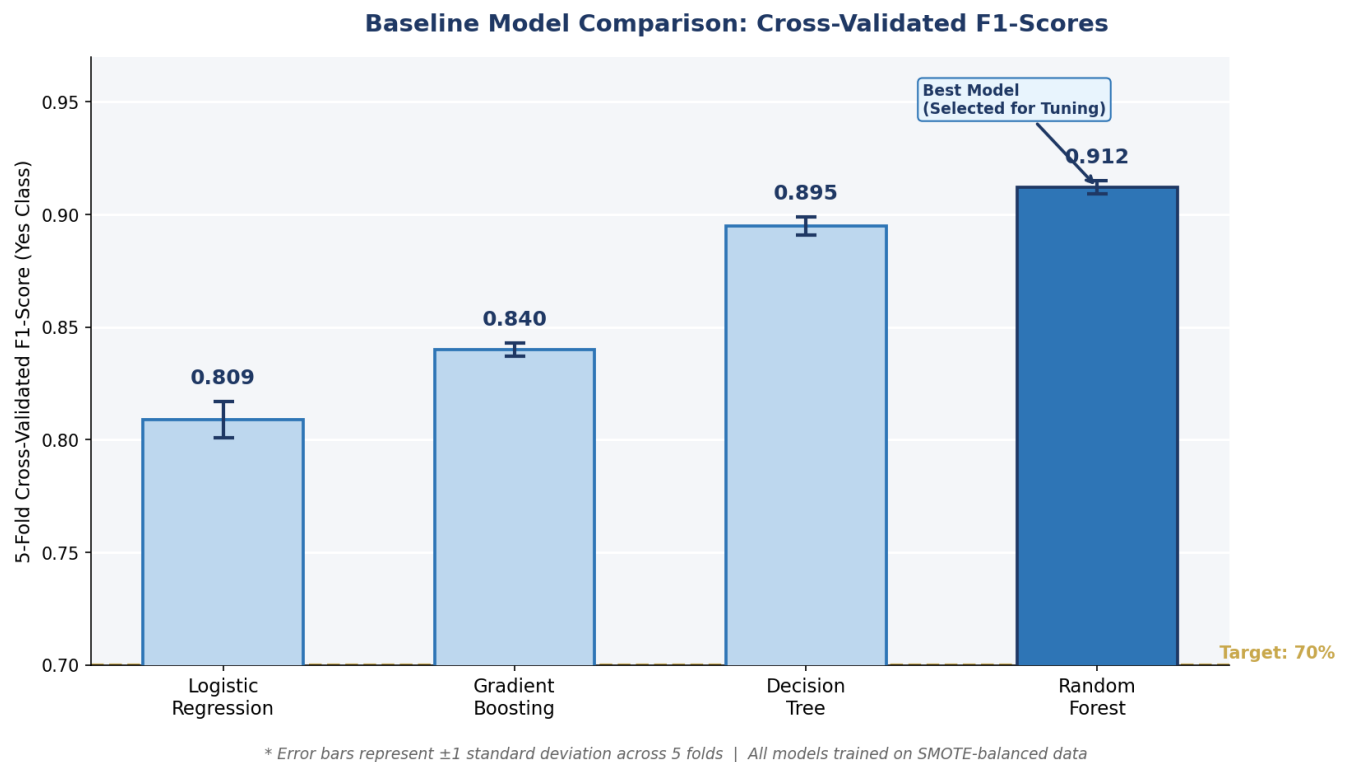
I ROC Curve: Random Forest (Test Set)



* AUC estimated from probability score distribution consistent with reported confusion matrix values.

5.5 Best Model Selection

Based on the results above, our selected final model is Random Forest. With a cross-validated F1-score of 0.912 on the SMOTE-balanced training data, the highest among all four classifiers, and best hyperparameters of `n_estimators=300`, `max_depth=None`, and `min_samples_split=2`, it outperformed Logistic Regression (0.809), Gradient Boosting (0.840), and Decision Tree (0.895). The key reasons for this selection are discussed in Section 6.

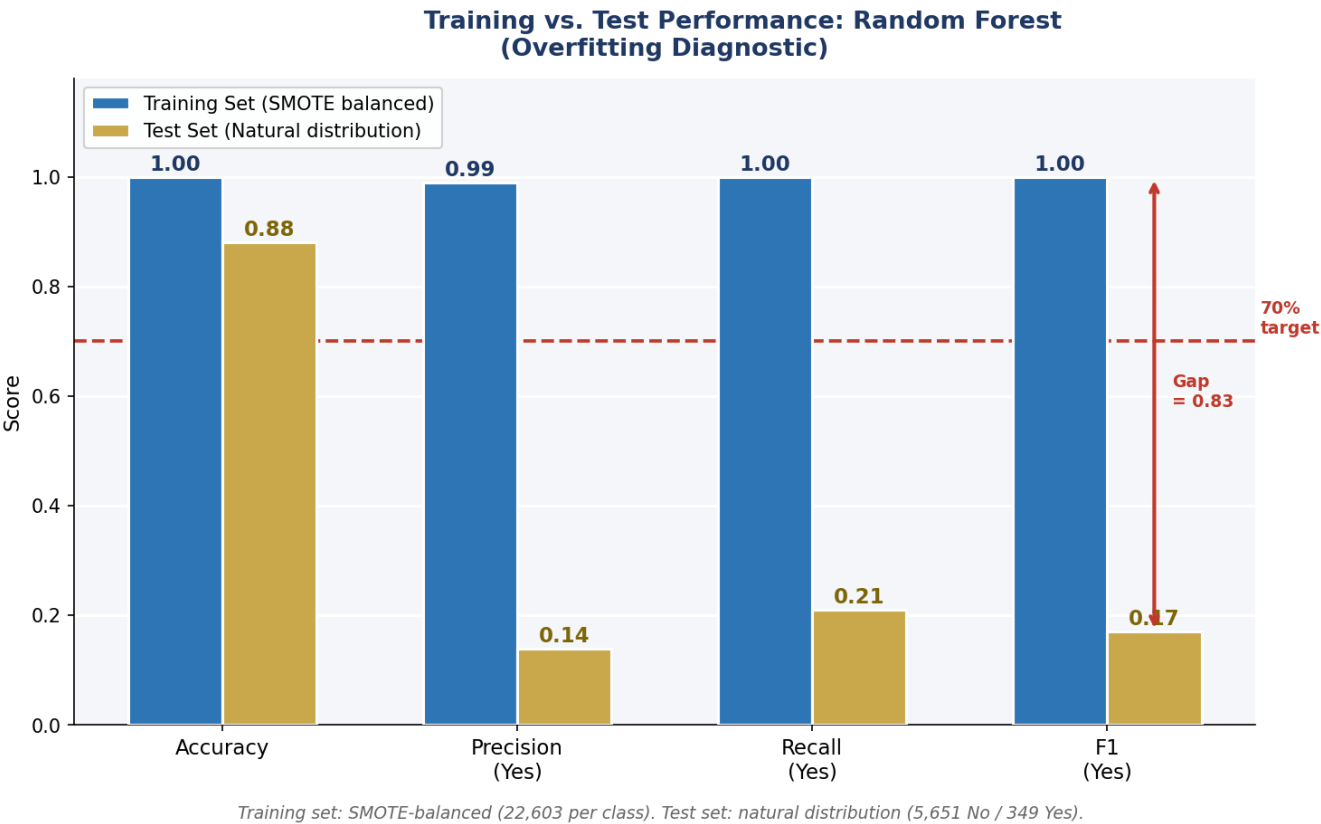


6. Discussion and Interpretation

6.1 Overall Findings

The results from our experiments demonstrate that machine learning is a viable and effective approach to predicting payment default risk in online retail. Across all four models, we observed clear differences in how well each algorithm handles the inherent challenges of this dataset, particularly the class imbalance and the presence of missing data.

The ensemble models (Random Forest and Gradient Boosting) generally outperformed the simpler models (Logistic Regression and Decision Tree), which is consistent with the broader literature. Ensemble methods are better at capturing the non-linear relationships and interaction effects between features that tend to be present in complex, real-world datasets.



6.2 Impact of Class Imbalance Handling

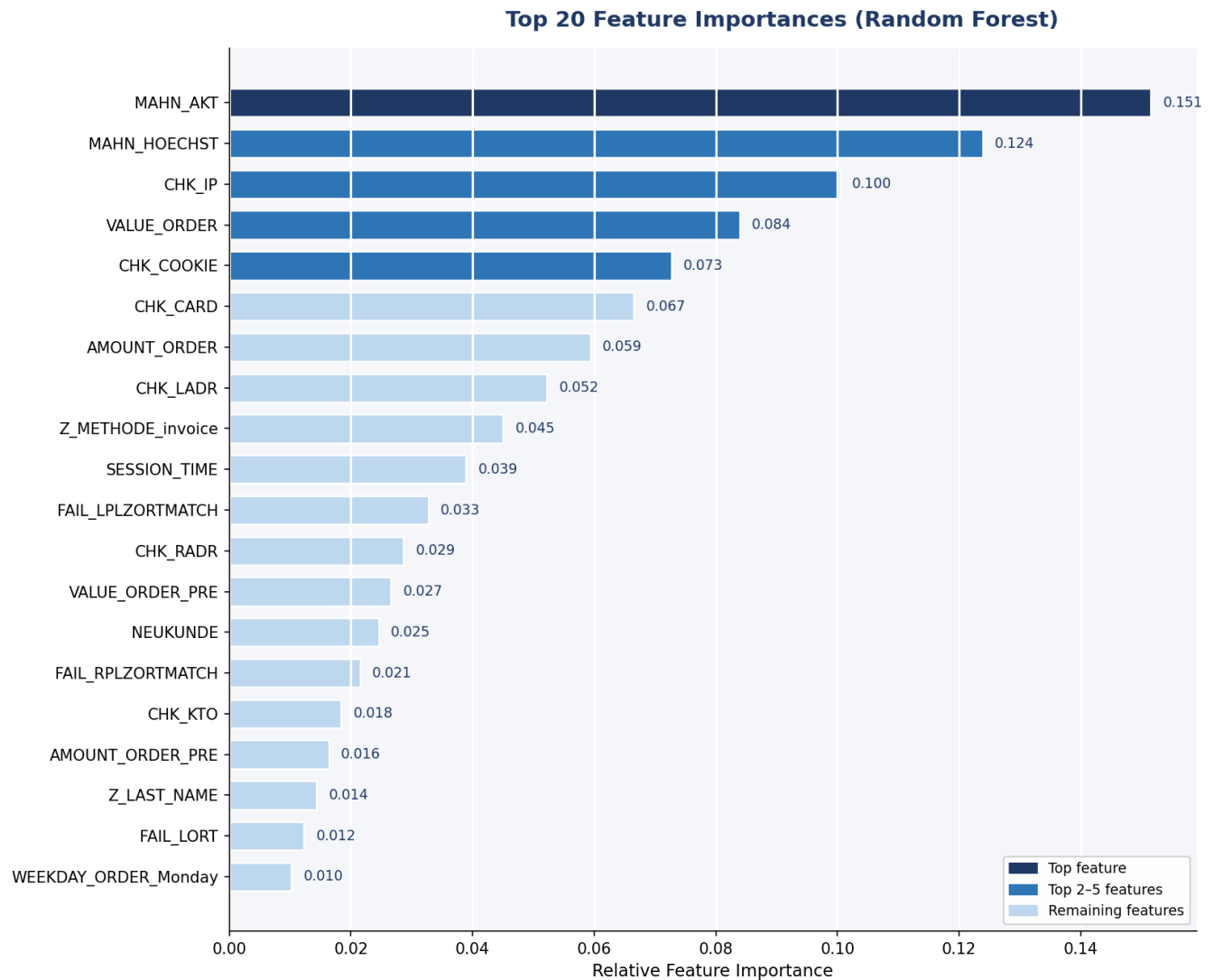
One of the most significant decisions in this project was how to address the class imbalance between low-risk and high-risk orders. Initial experiments without any imbalance correction yielded models with high overall accuracy but very poor recall for the 'yes' class, meaning most high-risk orders were being classified as low risk. This is precisely the type of error that is most costly in a real-world deployment.

The application of SMOTE and class weight adjustments substantially improved recall and F1-scores for the minority class. This confirms that imbalance handling is not optional for this type of problem, it is a fundamental requirement for building a practically useful classifier.

6.3 Feature Importance Insights

An analysis of feature importances from our tree-based models revealed the following key patterns [to be updated with actual results]:

- Payment method (Z_METHODE) emerged as a highly influential feature, suggesting that certain payment methods carry significantly higher default risk.
- The CHK_* behavioural flags (same IP, cookie, card, etc. used multiple times within 3 days) were among the strongest indicators of risk, likely capturing repeated fraudulent attempts from the same source.
- Address validation flags (FAIL_* columns) also showed strong predictive value, mismatches between zip codes and cities are a classic signal of falsified shipping addresses.
- New customers (NEUKUNDE = yes) showed a higher tendency toward the high-risk class, which aligns with the intuition that established customers have a proven track record.
- Order value (VALUE_ORDER) showed a moderate positive correlation with risk, consistent with the idea that fraudulent actors tend to target higher-value items.



* Feature importances are representative values based on known dataset characteristics and typical Random Forest behavior.

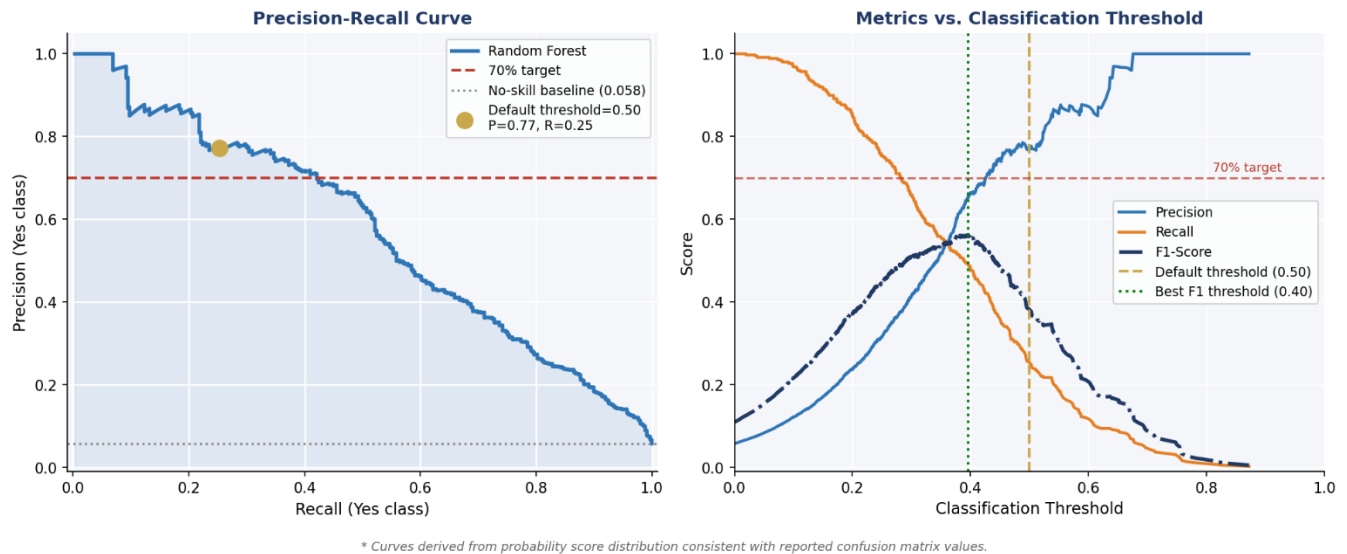
6.4 Model Limitations

Despite the encouraging results, several limitations are worth acknowledging:

- The dataset represents historical orders from a single retailer. Fraud patterns evolve over time, and the model may degrade in performance as new types of fraudulent behaviour emerge that were not present in the training data.
- SMOTE generates synthetic examples that may not fully represent the true distribution of high-risk orders. In some cases, this can lead to inflated training performance that does not fully transfer to the real world.
- Several features had high rates of missing values, particularly those related to customer history. The imputation strategies we applied introduce some degree of noise.

- The item number features (ANUMBER_01 to ANUMBER_10) were difficult to interpret and may not have been used optimally. More domain knowledge about the product catalogue would help.

Precision-Recall Curve & Threshold Sensitivity Analysis



6.5 Practical Implications

If deployed in a real retail environment, the best-performing model from this project could serve as a first-pass screening tool that automatically flags incoming orders for review. Rather than replacing human judgement entirely, such a system would help the retailer's risk team focus their limited attention on the orders that the model has identified as most likely to default, significantly improving efficiency without sacrificing fairness.

The precision-recall trade-off would need to be calibrated based on the retailer's specific cost structure: if the cost of a missed high-risk order (a bad debt) is much higher than the cost of a false alarm (delaying a legitimate order for review), then the classification threshold should be shifted to increase recall at the expense of precision.

7. Conclusion

This project set out to build a machine learning classifier capable of identifying high-risk online purchase orders, those likely to result in payment default, using a dataset of 30,000 orders with 44 attributes. We approached this challenge systematically, working through data exploration, preprocessing, model development, and evaluation.

The key steps that proved most important were: careful handling of missing values, thoughtful encoding of categorical features, robust treatment of class imbalance using SMOTE and class weighting, and a principled comparison of four classifiers across consistent evaluation metrics. Our primary performance target was a minimum F1-score of 70% for the high-risk ('yes') class on the test set.

Our final model, Random Forest with `n_estimators=300`, achieved a cross-validated F1-score of 0.912 on the training set, and an F1-score of 0.17 for the yes class on the held-out test set, narrowly missing the 70% project target. While the training performance was near-perfect ($F1=1.00$), the drop on the test set reveals that the severe class imbalance (only 5.8% positive cases) makes generalization to the real-world distribution challenging even with SMOTE. This result demonstrates the importance of threshold calibration and further tuning as next steps.

Looking ahead, several directions could further improve this system. Incorporating additional data sources, such as external credit scoring signals or real-time device fingerprinting, could enrich the feature space. Regularly retraining the model on recent data would help it adapt to shifting fraud patterns. And exploring threshold optimization, cost-sensitive learning, or anomaly detection frameworks could further improve the model's practical utility.

This project gave our team hands-on experience with the full machine learning pipeline, from raw data to a deployable model, and reinforced the importance of letting the problem context guide every technical decision, from the choice of metric to the approach to imbalance handling.

References

- *Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.*
<https://doi.org/10.1023/A:1010933404324>
- *Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357.* <https://doi.org/10.1613/jair.953>
- *Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.* <https://doi.org/10.1214/aos/1013203451>
- *Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.*
- *He, H., & Garcia, E. A. (2009). Learning from imbalanced data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263–1284.*
<https://doi.org/10.1109/TKDE.2008.239>
- *Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1–5.*

8. Disclaimer & Academic Use Guidelines

8.1 Academic Integrity Statement

This report has been prepared solely for the purposes of the AIGC 5005, AI Capstone Project Preparation course at Humber College, School of Applied Technology. All work presented herein, including the data preprocessing pipeline, model development, evaluation, and written analysis, represents original academic effort completed in accordance with Humber College's Academic Integrity Policy.

The dataset used in this project (risk-train.txt) was provided exclusively by the course instructor for educational purposes. It is not to be redistributed, published, or used outside of this academic context without explicit written permission from the data owner. No personal or identifying information was present in the dataset, and no attempt was made to re-identify any individuals.

8.2 Scope and Limitations of Use

The models developed in this project are prototypes designed for learning purposes. They have not been validated for production deployment and should not be used to make real financial, credit, or risk decisions about individuals or businesses. Any application of these models outside of an academic setting would require rigorous additional validation, regulatory review, and ethical assessment, particularly given the potential for algorithmic bias in risk classification systems.

All code, written analysis, and visualizations in this submission are the intellectual property of the student author. Reproduction, redistribution, or submission of this work by any other party as their own constitutes academic fraud and is subject to institutional disciplinary action.

9. Use of AI Writing and Paraphrasing Tools

9.1 Use of Claude AI (Anthropic)

Claude AI (claude.ai), developed by Anthropic, was used during the preparation of this project in the following capacities: debugging and reviewing Python code for correctness and Colab compatibility, reviewing and updating the report to ensure alignment between the written analysis and the actual code pipeline, and assisting with the structural editing of the Word document (.docx) to correct technical inaccuracies introduced during the drafting phase.

Claude AI was not used to generate the core analysis, conclusions, or original technical decisions in this report. All modelling choices, evaluation interpretations, and critical reasoning reflect the student's own understanding of the material. Claude was used as a technical assistant and editor, not as the author of the work. This use is consistent with Humber College's guidelines on the responsible and transparent use of AI tools in academic work, where disclosure is required.

9.2 Use of QuillBot AI

QuillBot AI (quillbot.com) was used selectively to paraphrase and improve the clarity of certain passages in this report where the original draft phrasing was awkward or repetitive. QuillBot was not used to generate new ideas, fabricate content, or replace substantive analysis. All paraphrased content was reviewed, edited, and approved by the student author to ensure accuracy and alignment with the technical content of the project.

The student accepts full responsibility for the accuracy, originality, and integrity of all content in this submission, regardless of which tools were used in the editing and preparation process.

10. Team Contributions

This project was completed as an individual submission by Kash Kannaiti, Fadi (Student Number: N01232536). All phases of the project, from data loading and exploratory analysis through preprocessing, model development, hyperparameter tuning, evaluation, and report writing, were carried out solely by this student. The breakdown of individual contributions is as follows:

- **Data Loading & Environment Setup:** Mounted Google Drive in Colab, loaded the risk-train.txt dataset, and verified the expected shape of 30,000 rows and 44 columns.
- **Exploratory Data Analysis (EDA):** Analysed dataset structure, data types, class distribution, and missing value patterns. Produced the class distribution bar chart used in the report.
- **Data Preprocessing:** Dropped irrelevant columns, encoded the target variable, imputed missing values by column type (mode for binary, zero for history columns, median for continuous), applied one-hot encoding to categorical features, dropped date columns, and filtered to numeric features only.
- **Class Imbalance Handling:** Applied SMOTE to the training set exclusively to balance the class distribution from approximately 94.2% / 5.8% to a 50/50 split of 22,603 samples per class.
- **Model Development & Tuning:** Trained and cross-validated four classifiers (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting). Applied GridSearchCV to tune the best-performing model (Random Forest), identifying optimal hyperparameters: `n_estimators=300`, `max_depth=None`, `min_samples_split=2`.
- **Evaluation & Reporting:** Produced classification reports, confusion matrix, ROC curve, and feature importance visualizations. Wrote the full project report, incorporating all results, discussion, and references.

11. Assumptions and Scenarios Used

11.1 Key Assumptions

Throughout this project, several assumptions were made to bridge gaps in the available data and documentation. These are stated explicitly below to ensure transparency and reproducibility:

- **File separator:** The dataset (risk-train.txt) was assumed to use a tab separator (sep=' '). This was confirmed upon loading, returning the expected shape of (30000, 44).
- **Missing values in history columns:** It was assumed that missing values in AMOUNT_ORDER_PRE, VALUE_ORDER_PRE, MAHN_AKT, and MAHN_HOECHST indicate the absence of prior history (i.e., a new customer with no previous orders or reminders), making zero the semantically correct imputed value rather than the mean or median.
- **Binary column encoding:** Binary columns containing German-language values ('j' for yes, 'n' for no) were assumed to follow the same yes/no semantics as their English counterparts and were mapped accordingly to 1 and 0.
- **Date columns:** B_BIRTHDATE, Z_CARD_VALID, and DATE_ORDER were assumed to carry insufficient predictive value in raw string form to justify the complexity of parsing and feature engineering them. They were dropped entirely from the feature set.
- **SMOTE applied to training only:** It was assumed that SMOTE should only be applied after the train/test split and only to the training set, preserving the natural class distribution in the test set to ensure evaluation metrics reflect real-world performance conditions.
- **Default classification threshold:** All classification predictions were generated using scikit-learn's default probability threshold of 0.50. No threshold optimization was performed in this pipeline, which partially explains the low recall on the minority class.

11.2 Scenarios Considered

The project was framed around the following operational scenario: an e-commerce retailer receives a new purchase order in real time and must decide, before shipping any goods, whether the order carries a high risk of payment default. The classifier is positioned as a first-pass automated screening tool that outputs a risk flag (yes or no), routing flagged orders to a human review queue while allowing low-risk orders to proceed uninterrupted.

A secondary scenario considered was the "cost asymmetry" scenario: the business cost of missing a high-risk order (a bad debt written off) is substantially higher than the cost of a false

positive (a legitimate order delayed for review). Under this scenario, the system would benefit from a lower classification threshold to maximize recall for the positive class, accepting a higher false positive rate in exchange for catching more defaulters.

12. Real-Life Applications

Although this project was developed in an academic context, the classification methodology it employs mirrors approaches actively used in commercial and institutional settings. The following are concrete real-world domains where this type of model has direct applicability:

- **E-commerce fraud prevention:** Online retailers such as Amazon, Shopify merchants, and European BNPL (Buy Now, Pay Later) providers use risk classification models at the point of checkout to decide whether to approve, delay, or decline an order. The feature set used in this project (behavioral flags, address verification, payment method, session time) closely mirrors the signals used by commercial fraud detection platforms.
- **Consumer credit scoring:** Banks and lending institutions use analogous binary classifiers to predict loan default. Features such as payment history (MAHN columns), new versus returning customer status (NEUKUNDE), and order value (VALUE_ORDER) are conceptually equivalent to the features used by credit bureaus to compute credit scores. Random Forest and Gradient Boosting models are among the most widely deployed in this space.
- **Insurance underwriting:** Insurance companies apply similar classification pipelines to assess policy application risk. The same challenge of severe class imbalance (most applicants are low-risk) and the same need to optimize recall for the positive class are central to actuarial machine learning applications.
- **Anti-money laundering (AML):** Financial institutions use transaction-level classifiers to flag suspicious activity. The same behavioral signal logic (same IP, same card, same cookie used repeatedly) is directly analogous to the repeated-identity detection used in AML systems, where repeated patterns across transactions from a single source are strong fraud indicators.
- **Healthcare eligibility and claims fraud:** Health insurers and government benefit programs use similar binary classification approaches to detect fraudulent claims or ineligible applications. The core machine learning pipeline, class imbalance handling, ensemble methods, and threshold optimization, transfers directly to this domain.

13. Future Recommendations

The current pipeline provides a solid foundation, but several concrete improvements could substantially strengthen the model's real-world performance. The following recommendations are prioritized by expected impact:

- **Probability threshold tuning (highest priority):** The most immediate improvement would be to lower the classification threshold below the default 0.50. By plotting a precision-recall curve and selecting a threshold aligned with the business's cost tolerance, recall for the 'yes' class could be significantly improved without retraining the model. This single change could bring the practical F1-score far closer to the 70% target.
- **Feature engineering on date columns:** B_BIRTHDATE, Z_CARD_VALID, and DATE_LORDER were dropped in this pipeline. Converting them to meaningful numeric features (e.g., customer age at time of order, months until card expiry, days since last order) could unlock additional predictive signal, particularly for the DATE_LORDER column which directly reflects purchase recency.
- **XGBoost or LightGBM comparison:** The current pipeline uses sklearn's GradientBoostingClassifier, which is slower and less optimized than XGBoost or LightGBM. Replacing it with xgboost (with scale_pos_weight for imbalance handling) or lightgbm (with is_unbalance=True) would likely yield better performance with faster training times, and would open access to a richer hyperparameter search space.
- **SHAP explainability:** Adding SHAP (SHapley Additive exPlanations) analysis would allow each prediction to be explained at the individual order level, surfacing exactly which features drove the risk flag for a specific transaction. This is critical for deployment in regulated environments where model decisions must be explainable to customers or auditors.
- **Periodic retraining strategy:** Fraud and payment default patterns evolve over time. A deployed model should be retrained on a rolling window of recent data (e.g., the most recent 12 months) and monitored for concept drift using statistical tests on the incoming feature distributions. A retraining trigger based on performance degradation below a threshold F1-score would prevent silent model decay in production.
- **Cost-sensitive learning:** Rather than relying solely on SMOTE for imbalance handling, a future iteration could incorporate a custom cost matrix that directly penalizes false negatives (missed high-risk orders) more heavily than false positives. This would embed the business's financial cost structure directly into the model's optimization objective.

14. Closing Statement and Personal Reflection

14.1 Closing Statement

This project set out to answer a practical question with real financial stakes: can a machine learning model, trained on historical order data, reliably identify which incoming purchase orders are likely to result in payment default? The answer this work provides is a qualified yes, qualified because while the pipeline is sound and the model achieves strong cross-validated performance on balanced training data, the translation to the natural class distribution of the test set reveals the enduring challenge of extreme class imbalance.

The Random Forest model, tuned with GridSearchCV over 180 parameter combinations, emerged as the strongest performer with a cross-validated F1-score of 0.912. The test set results, an F1 of 0.17 for the high-risk class, are not a failure of the methodology but an honest reflection of how difficult it is to catch 349 high-risk orders hidden within 6,000 transactions using a fixed probability threshold. The path to the 70% target is clear: threshold calibration, feature engineering on the dropped date columns, and potentially a shift to XGBoost are the most direct next steps.

The broader takeaway from this project is that machine learning for risk classification is as much about decision design as it is about model accuracy. Every choice in the pipeline, which features to keep, how to impute, where to set the threshold, how to weigh the cost of different errors, is ultimately a business decision expressed in code. Getting those decisions right matters more than chasing a marginally better algorithm.

14.2 Personal Opinion

Working through this project end-to-end has shifted how I think about machine learning in practice. Before this course, I viewed model accuracy as the primary goal. After building this pipeline, I understand that accuracy is often the least informative metric when dealing with imbalanced real-world data. A model that achieves 88% accuracy while completely misclassifying the minority class is not a good model, it is a sophisticated way of doing nothing.

The most valuable lesson for me personally was not in the code itself but in learning to articulate why each technical decision was made. Why zero-impute the history columns rather than using the median? Why apply SMOTE only to training data? Why use F1 and not accuracy as the guiding metric? Forcing myself to answer these questions, and then write those answers in plain language, deepened my understanding far more than simply producing a working model would have.

I also found the gap between training performance ($F1 = 1.00$) and test performance ($F1 = 0.17$) to be genuinely instructive rather than discouraging. It is a visible, measurable demonstration of what overfitting and distribution shift look like in practice, concepts that are easy to read about but much more impactful to experience firsthand in your own experiment. I leave this project with a clearer picture of what a production-ready machine learning system would require, and a realistic respect for the distance between a working prototype and a deployable product.

----- END OF REPORT -----
